

Dynamic Programming in a Door & Key Environment

1st Armon Barakchi
 abarakchi@ucsd.edu
 A16950466

I. INTRODUCTION

In the field of robotics, the natural next step after making observations is movement. To that end, optimal control policies are needed to achieve the robot's goal most efficiently. The natural modeling choice for the robot's observable space and action space is a *Markov Decision Process* (MDP). This framework allows the problem to be expressed as a sequence of decisions, where at each timestep the agent observes its current state, selects an action, and transitions to a new state while incurring a cost. This is the cost that we want to design and minimize to lead to desired behavior.

Dynamic Programming (DP) is the tool of choice for minimizing the value function of MDPs when the environment is fully known. DP computes the optimal *cost-to-go* for every possible state by working backwards from the goal. The result is a complete control policy that guarantees minimum total cost regardless of where the agent starts.

This project applies DP to the **Door and Key** environment [1], a grid-world in which an agent must navigate to a goal, potentially collecting a key and unlocking a door along the way. The environment is formulated as an MDP and solved under two scenarios: a set of known fixed maps, and a family of structurally consistent random maps. In both cases, the goal is to compute an optimal policy that minimizes the total cost of reaching the goal.

II. PROBLEM FORMULATION

A. Environment Setup

A **Door and Key** environment [1] is an $N \times N$ grid where each element denoted (r, c) is either accessible space, inaccessible space, a door, or the final objective. Additionally, this door may be locked or unlocked, and in the former case a key (which may be located at any accessible space) must be used to unlock it.

There is an agent in this environment with observable fully observable state. The goal of the agent is to get from the starting point to the final objective optimally. The agent has the following available actions:

- 1) Move Forward - MF
- 2) Turn Left - TL
- 3) Turn Right - TR
- 4) Pick Up Key - PK
- 5) Unlock Door - UD

In our setup, there are two types of maps: random and deterministic. There are seven "known" maps that all require different optimal control policies. There are 36 "random" maps that are all 10×10 and a single control policy must be developed to handle all of them. The restrictions of the random maps are as such:

Note: Let $(0,0)$ denote the top left, $(0,9)$ as bottom left, top right as $(9,0)$, and bottom right as $(9,9)$.

- 1) There is a vertical wall at column 5 with two doors at $(5,3)$ and $(5,7)$. Each door can either be open or unlocked.
- 2) The key is randomly located at $(2,2), (2,3), (1,6)$.
- 3) The goal is randomly located at $(6,1), (7,3), (6,6)$.
- 4) The agent starts at $(4,8)$ facing up.

B. MDP Formulation

This game can be formulated as a *Markov Decision Process* by defining all of the components [2] of an MDP. Below, the tuple $(\mathcal{X}, \mathcal{U}, p_0, p_f(\cdot | \mathbf{x}_t, \mathbf{u}_t), T, \ell(x, u), q(x), \gamma)$ is defined, fully specifying the MDP.

• State Space:

- *Known Maps*: $x = (r, c, d, k, \delta_1)$
- *Random Maps*: $x = (r, c, d, k, \delta_1, \delta_2, s_K, s_G)$

where $(r, c) \in \{0, \dots, N-1\}^2$ is the agent's grid position, $d \in \{\text{up, down, left, right}\}$ is the heading direction, $k \in \{0, 1\}$ is a boolean indicating whether the agent currently holds the key, $\delta_j \in \{0, 1\}$ indicates whether door j is unlocked, and in the unknown map case, s_K, s_G denote the key and goal positions.

• Control Space: $\mathcal{U} = \{\text{MF, TR, TL, PK, UD}\}$

• Prior Distribution:

- *Known Map*:

$$p_0(x) = \begin{cases} 1, & x = x_{\text{start}} \\ 0, & \text{otherwise} \end{cases}$$

where x_{start} has fixed position, orientation, no key ($k = 0$), and known door states.

- *Random Map*:

$$p_0(x) = p_0(r, c, d, k, \delta_1, \delta_2, s_K, s_G)$$

where

- * $(r, c, d) = (4, 8, \uparrow)$ is deterministic,
- * $k = 0$ (agent does not initially hold the key),

- * $s_K \in \{(2, 2), (2, 3), (1, 6)\}$ is assumed to be uniformly distributed over 3 positions,
- * $s_G \in \{(6, 1), (7, 3), (6, 6)\}$ is assumed to be uniformly distributed over 3 positions,
- * $\delta_1, \delta_2 \in \{0, 1\}$ are each assumed to be uniformly distributed.

Assuming independence across all random variables, there are $3 \times 3 \times 4 = 36$ equally likely initial configurations, so

$$p_0(x) = \frac{1}{36}, \quad \forall x \in \mathcal{X}_{\text{init}}.$$

- **Motion Model** $f(x_t, u_t)$:

The deterministic transition function $f : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$ is defined action-by-action. Let $x_t = (r, c, d, k, \delta_1, \delta_2, \dots)$. Then $x_{t+1} = f(x_t, u_t)$ is given by:

- **Turn Left (TL) / Turn Right (TR):** The position (r, c) is unchanged; d rotates 90° counterclockwise or clockwise respectively. All other state components are unchanged.
- **Move Forward (MF):** Let (r', c') be the cell directly ahead in direction d . If (r', c') is a free cell (not a wall, and not a locked door), then $(r, c) \leftarrow (r', c')$. Otherwise the agent remains in place and the cost is applied regardless of whether or not the movement succeeds. All other state variables remain unchanged.
- **Pick Up Key (PK):** If the agent is standing adjacent to the key location ($s_K = (r, c)$), facing the key, and does not yet hold the key ($k = 0$), then $k \leftarrow 1$. Otherwise the state is unchanged. The cost is incurred regardless of whether or not the action succeeds.
- **Unlock Door (UD):** If the cell directly ahead in direction d is a locked door j ($\delta_j = 0$) and the agent holds the key ($k = 1$), then $\delta_j \leftarrow 1$. Otherwise the state is unchanged. The cost is incurred regardless of whether or not the action succeeds.

- **Conditional State Transition** $p_f(x_{t+1} | x_t, u_t)$:

Since the environment is deterministic once the map is fixed,

$$p_f(x_{t+1} | x_t, u_t) = \begin{cases} 1, & x_{t+1} = f(x_t, u_t), \\ 0, & \text{otherwise.} \end{cases}$$

- **Time Horizon** $T = \infty$:

Both scenarios are formulated as infinite-horizon problems. Convergence of the value function is guaranteed because all goal states are absorbing, all stage costs are strictly positive ($\ell(x, u) > 0$), and every state can reach the goal in finite steps under some policy [3].

- **Stage Cost:**

$$\ell(x, u) = c(u) > 0 \quad \forall u \in \mathcal{U}.$$

Action	Cost
Move Forward (MF)	3
Turn Left (TL)	1
Turn Right (TR)	1
Pick Up Key (PK)	2
Unlock Door (UD)	5

TABLE I: Stage costs for each action in the Door & Key environment.

- **Terminal Cost:**

$$q(x) = \begin{cases} 0, & x \in \mathcal{X}_{\text{goal}} \\ \infty, & \text{otherwise} \end{cases}$$

- **Discount Factor:** $\gamma = 1$.

C. Main Problem Formulation

Now that the Door and Key environment has been formulated as an MDP, we can also define the central control problem: we wish to determine a policy that minimizes the total accumulated cost required for the agent to reach the goal state.

To this end, we introduce the value function, which quantifies the minimum cost-to-go from a given state under a specified policy.

Value function under a policy π :

For a policy $\pi : \mathcal{X} \rightarrow \mathcal{U}$, define the cost-to-go from state x as

$$V^\pi(x) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \ell(x_t, \pi(x_t)) \mid x_0 = x \right].$$

Since the Door & Key environment is deterministic once the map is fixed, this reduces to the total accumulated cost along the trajectory induced by π .

Optimal value function:

The optimal value function is the minimum achievable total cost-to-go from state x :

$$V^*(x) = \min_{\pi} V^\pi(x) = \min_{\pi} \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \ell(x_t, \pi(x_t)) \mid x_0 = x \right].$$

Objective problem:

Our objective is to find a policy π^* such that

$$\pi^* \in \arg \min_{\pi} \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \ell(x_t, \pi(x_t)) \mid x_0 \sim p_0 \right].$$

Equivalently, for each initial state x , we seek to solve

$$V^*(x) = \min_{u_0, u_1, \dots} \sum_{t=0}^{\infty} \ell(x_t, u_t)$$

subject to

$$x_{t+1} = f(x_t, u_t), \quad x_0 = x.$$

For known maps, x_0 is deterministic. In contrast, in the random map setting, $x_0 \sim p_0$ captures the randomness in the environment configuration, and the objective minimizes the expected cost over this distribution.

Optimality equation:

Since the problem is an infinite-horizon shortest-path problem with absorbing goal states, the optimal value function satisfies

$$V^*(x) = \min_{u \in \mathcal{U}} \{ \ell(x, u) + V^*(f(x, u)) \}, \quad x \notin \mathcal{X}_{\text{goal}}.$$

For goal states, we impose the boundary condition

$$V^*(x) = 0, \quad x \in \mathcal{X}_{\text{goal}}.$$

Optimal policy:

Once V^* is obtained, the optimal policy is given by

$$\pi^*(x) \in \arg \min_{u \in \mathcal{U}} \{ \ell(x, u) + V^*(f(x, u)) \}.$$

III. TECHNICAL APPROACH

We solve the problem posed in Section II-C using *Dynamic Programming*. Dynamic Programming relies on one central idea – Bellman’s **Principle of Optimality** [4]: any suffix of an optimal trajectory is optimal for the subproblem starting at that state. This means the optimal policy that solves the objective problem can be recovered by solving a sequence of local one-step minimizations backwards in time. Specifically, if V^* satisfies the Bellman optimality equation

$$V^*(x) = \min_{u \in \mathcal{U}} \{ \ell(x, u) + V^*(f(x, u)) \}, \quad x \notin \mathcal{X}_{\text{goal}},$$

then the policy

$$\pi^*(x) = \arg \min_{u \in \mathcal{U}} \{ \ell(x, u) + V^*(f(x, u)) \}$$

achieves $V^*(x_0)$ from any initial state x_0 , which is precisely the minimum total cost posed in the objective problem of Section II-C.

Both scenarios share the same DP algorithm and differ only in how the state space $\mathcal{X}$ is constructed.

A. Infinite-Horizon Backward DP

The DP algorithm follows the lecture formulation directly. The standard finite-horizon algorithm initializes $V_T(x) = q(x)$ and sweeps backwards from $t = T - 1$ to $t = 0$. Since our formulation has an infinite horizon ($T = \infty$) as established in Section II-B, the horizon is not known in advance. We therefore adapt the algorithm by repeating backward sweeps until the value function converges. Each sweep uses V_{t+1} from the previous sweep to compute V_t which is consistent with the standard finite-horizon formulation. Convergence to the true V^* is guaranteed because all stage costs are strictly positive ($\ell(x, u) > 0$), goal states are absorbing, and every reachable state can reach the goal in finite steps.

1) *Phase 1: State Space Enumeration*: Since the state space $\mathcal{X}$, transition model f , and stage costs ℓ are not all available upon construction of the environment, the environment must be simulated to construct an explicit MDP graph prior to running DP. This is done using the breadth-first search shown in Algorithm 1.

Algorithm 1 BFS State Space Enumeration

Require: Initial environment env_0 , info

Ensure: Transition table $\text{succ}[x][u] = (x', c)$, state map visited

```

0:  $x_0 \leftarrow \text{STATEKEY}(\text{env}_0, \text{info})$ 
0:  $\text{queue} \leftarrow \{x_0\}$ ,  $\text{visited} \leftarrow \{x_0 \mapsto \text{env}_0\}$ ,  $\text{succ} \leftarrow \{\}$ 
0: while queue is not empty do
0:    $x \leftarrow \text{queue.pop}()$ 
0:   for  $u \in \mathcal{U}$  do
0:      $(x', c) \leftarrow \text{SIMULATE}(\text{visited}[x], u)$ 
0:      $\text{succ}[x][u] \leftarrow (x', c)$ 
0:     if  $x' \notin \text{visited}$  then
0:        $\text{visited}[x'] \leftarrow \text{env after simulating } u$ 
0:        $\text{queue.push}(x')$ 
0:     end if
0:   end for
0: end while
0: return visited,  $\text{succ} = 0$ 

```

This produces the complete reachable state space $\mathcal{X}$ and transition table $\text{succ}[x][u] = (x', c)$ where $x' = f(x, u)$ and $c = \ell(x, u)$, which are required as input to the DP algorithm.

2) *Phase 2: Backward DP Sweep*: After constructing the explicit MDP graph, Algorithm 2 applies the DP algorithm [4]. It initializes the terminal cost $V_T(x) = q(x)$ and repeatedly sweeps backwards across time. At each sweep, V_t is computed from V_{t+1} which was obtained at the previous sweep. Sweeps continue until the maximum change δ falls below threshold θ , at which point $V_t \approx V^*$ and the algorithm has converged.

Algorithm 2 Infinite-Horizon Backward DP

Require: State set $\mathcal{X}$, transition table succ , goal set $\mathcal{X}_{\text{goal}}$, threshold θ

Ensure: Optimal value function V^* , optimal policy π^*

```

0:  $V(x) \leftarrow q(x) \forall x \in \mathcal{X}$ 
0: repeat
0:    $\delta \leftarrow 0$ 
0:   for  $x \in \mathcal{X}$  do
0:     if  $x \in \mathcal{X}_{\text{goal}}$  then
0:        $V_{\text{new}}(x) \leftarrow 0$ ,  $\pi^*(x) \leftarrow \text{None}$ 
0:     continue
0:     end if
0:     for  $u \in \mathcal{U}$  do
0:        $(x', c) \leftarrow \text{succ}[x][u]$ 
0:        $Q_t(x, u) \leftarrow c + V(x')$ 
0:     end for
0:      $V_{\text{new}}(x) \leftarrow \min_{u \in \mathcal{U}} Q_t(x, u)$ 
0:      $\pi^*(x) \leftarrow \arg \min_{u \in \mathcal{U}} Q_t(x, u)$ 
0:      $\delta \leftarrow \max(\delta, |V_{\text{new}}(x) - V(x)|)$ 
0:   end for
0:    $V \leftarrow V_{\text{new}}$ 
0: until  $\delta < \theta$ 
0: return  $V^* \leftarrow V$ ,  $\pi^* = 0$ 

```

Once $\delta < \theta$, the converged $V \approx V^*$ satisfies the optimality

equation. Executing π^* from any initial state x_0 achieves the minimum total cost

$$V^*(x_0) = \sum_{t=0}^{\infty} \ell(x_t, \pi^*(x_t)),$$

which is the solution to the objective problem posed in Section II-C.

3) *Phase 3: Policy Execution*: Once π^* is computed in Algorithm 2, Algorithm 3 produces the optimal action sequence by executing the policy in a greedy manner from x_0 . At each step, the agent observes its current state, selects $u = \pi^*(x)$, and transitions to $x' = f(x, u)$. By the Principle of Optimality, following π^* greedily at every step is optimal, so the resulting sequence achieves the minimum of the value function.

Algorithm 3 Policy Execution

Require: Initial state x_0 , optimal policy π^* , goal set $\mathcal{X}_{\text{goal}}$

Ensure: Optimal action sequence $\mathbf{u}^* = [u_0, u_1, \dots]$

```

0:  $x \leftarrow x_0$ ,  $\mathbf{u}^* \leftarrow []$ 
0: while  $x \notin \mathcal{X}_{\text{goal}}$  do
0:    $u \leftarrow \pi^*(x)$ 
0:    $\mathbf{u}^*.\text{append}(u)$ 
0:    $x \leftarrow f(x, u)$ 
0: end while
0: return  $\mathbf{u}^* = 0$ 
```

B. Part A: Known Maps

In the known maps, the environment is fully specified and the state is defined as $x = (r, c, d, k, \delta_1)$ as in Section II-B. For each of the seven maps a separate optimal policy π^* is computed by running the full pipeline: Algorithm 1 followed by Algorithm 2 followed by Algorithm 3 — on each map. The BFS is seeded from the single fixed initial state in each map, and since each map has a unique layout, the reachable state space differs per environment — resulting in a policy that is unique to that map.

C. Part B: Random Maps — Single Policy via State Augmentation

In the random map case, a single policy must address all 36 possible environments. We use state augmentation to address this constraint.

1) *State Augmentation*: The random map variables — key position s_K , goal position s_G , and door states δ_1, δ_2 — are observable at runtime and are therefore valid state components, which we have chosen to include making our state:

$$x = (r, c, d, k, \delta_1, \delta_2, s_K, s_G)$$

as defined in Section II-B. Under this augmented state, the Markov property holds across all 36 configurations so a single stationary policy $\pi^*(x)$ is well-defined and sufficient for the entire family of maps.

2) *Unified BFS and Backward DP*: Algorithm 1 is modified to seed the BFS from all 36 initial states simultaneously, building a single unified transition graph over the augmented state space:

$$\mathcal{X}_{\text{unified}} = \bigcup_{i=1}^{36} \mathcal{X}_i.$$

Algorithm 2 is then applied unchanged to $\mathcal{X}_{\text{unified}}$, producing a single converged V^* and policy π^* which is saved to disk. At runtime, the agent observes its full augmented state $x = (r, c, d, k, \delta_1, \delta_2, s_K, s_G)$ and executes $\pi^*(x)$ directly. This satisfies the single-policy requirement while achieving the minimum expected cost over p_0 .

3) *Handling Missing Perimeter Walls*: In the random map environments, there are no perimeters defined surrounding the grid. Any action which, when executed, would access a cell outside the grid's bounds is treated as a no-op: the agent's state remains unchanged, but the stage cost is still incurred. To this end, at each action we must check if the action is valid.

IV. RESULTS

A. Part A: Known Maps

The aforementioned pipeline was applied independently to each of the seven environments. First, the BFS enumerated the reachable state space, successfully creating an explicit MDP graph, and the backward sweep converged to the optimal value function, which generates a policy that minimizes the total accumulated cost from each initial state.

Table II summarizes the optimal cost achieved by π^* on each environment. The optimal action sequences are visualized as GIF strips in Appendix A.

Environment	Optimal Cost	Sequence Length
doorkey-5x5-normal	28	13
doorkey-6x6-normal	14	33
doorkey-6x6-direct	5	13
doorkey-6x6-shortcut	6	15
doorkey-8x8-normal	54	21
doorkey-8x8-direct	17	7
doorkey-8x8-shortcut	19	8

TABLE II: Optimal costs and action sequence lengths for each known environment under policy π^* .

In all environments, the results were consistent with expectations and the GIF strips serve as a visual representation of what the optimal trajectory is in these simple cases.

B. Part B: Random Maps

In the random map case, the BFS was seeded from all 36 initial configurations simultaneously, producing a unified MDP graph: $\mathcal{X}_{\text{unified}}$. The pipeline described in Section III converged, yielding a single stationary policy over the new state space.

The policy was successfully tested on all the random environments and the GIF strips are also shown in Appendix A. The agent's behavior appropriately varied for all permutations of the random map — consistently identifying the proper route.

V. CONCLUSION

This project formulated the Door & Key navigation problem as an infinite-horizon MDP and solved it using Dynamic Programming. Since the horizon T is not known in advance, the backward DP sweep was repeated until the value function converged, which is guaranteed for this problem as all stage costs are strictly positive and goal states are absorbing.

The algorithm was applied to two scenarios: seven known maps each requiring a dedicated optimal policy, and a family of 36 random maps requiring a single unified policy. In both cases, BFS was used to construct the explicit MDP graph from the black-box simulator, after which the backward sweep converged to V^* and π^* . For the random map scenario, state augmentation preserved the Markov property across all 36 configurations, reducing the problem to a single application of DP over the unified augmented state space. The resulting policies achieved the minimum total cost on all environments tested.¹

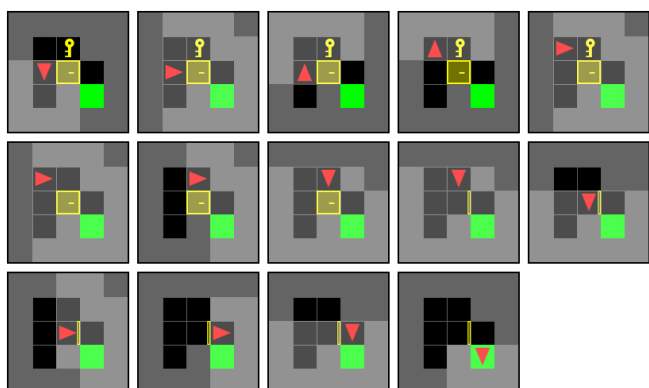
REFERENCES

- [1] M. Chevalier-Boisvert, B. Dai, M. Towers, R. Perez-Vicente, L. Willems, S. Lahlou, S. Pal, P. S. Castro, and J. Terry, “Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks,” 2023.
- [2] N. Atanasov, “Lecture 3: Markov Decision Processes,” 2025. ECE276B: Planning & Learning in Robotics.
- [3] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, vol. 1. Athena Scientific, 1995.
- [4] N. Atanasov, “Lecture 4: The Dynamic Programming Algorithm,” 2025. ECE276B: Planning & Learning in Robotics.

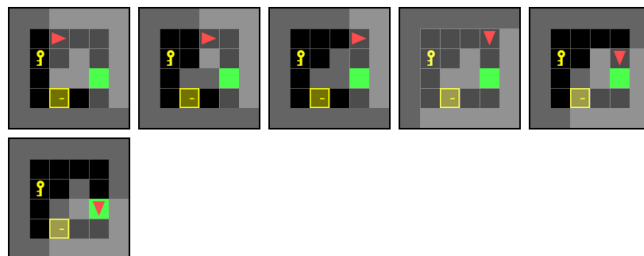
¹ChatGPT (OpenAI) was used as a clarification tool during this project as well as for help with LaTeX formatting issues.

APPENDIX

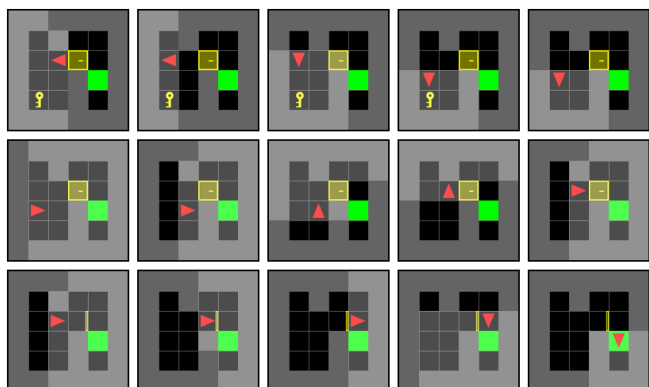
doorkey-5x5-normal



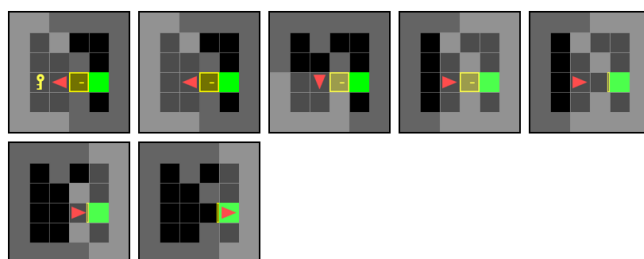
doorkey-6x6-direct



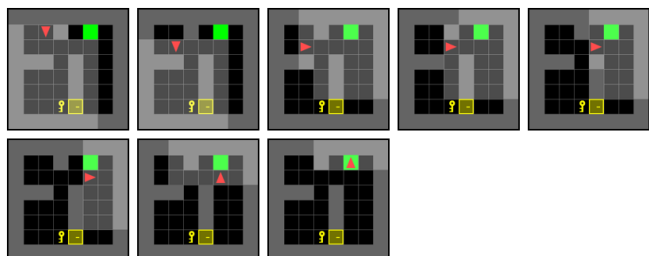
doorkey-6x6-normal



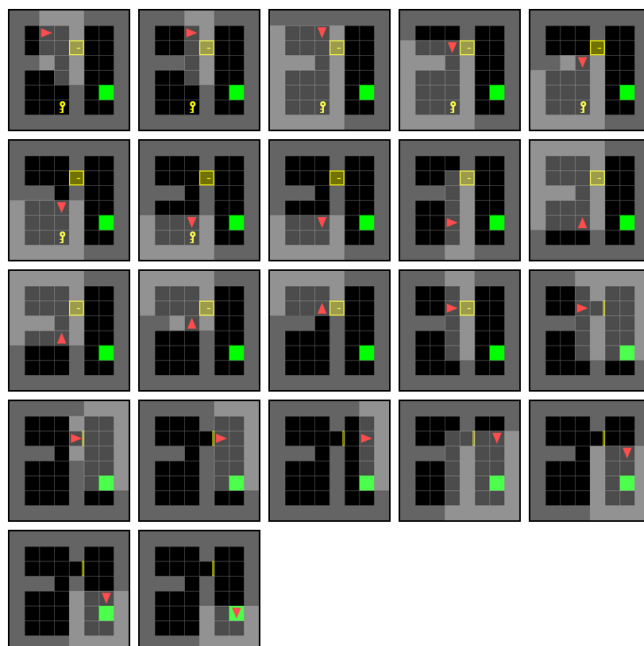
doorkey-6x6-shortcut



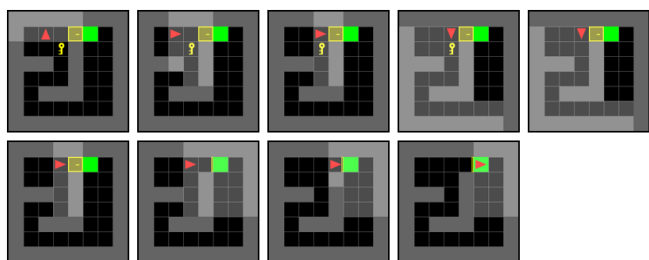
doorkey-8x8-direct



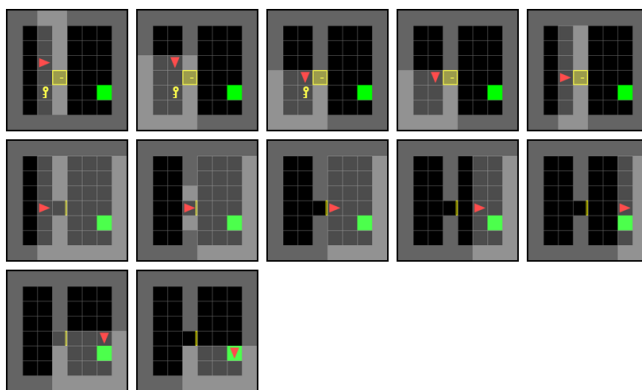
doorkey-8x8-normal



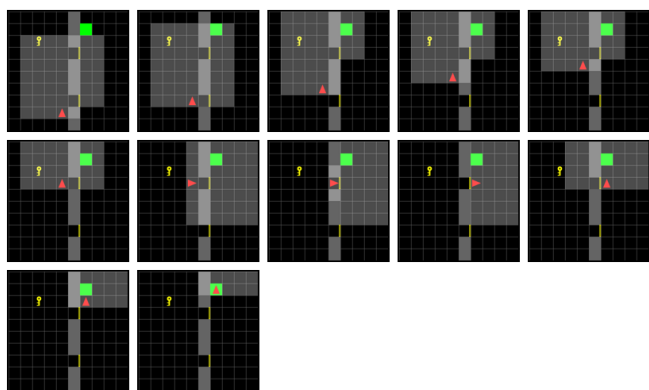
doorkey-8x8-shortcut



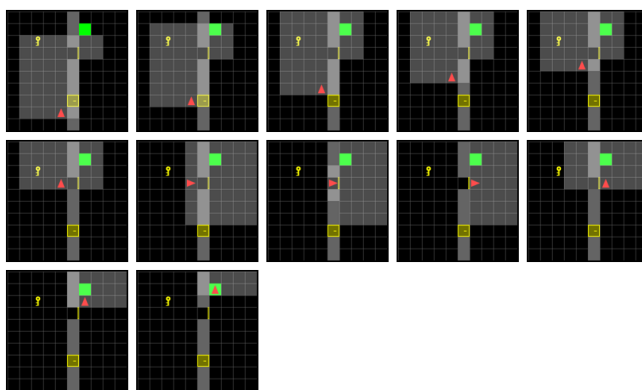
example-8x8



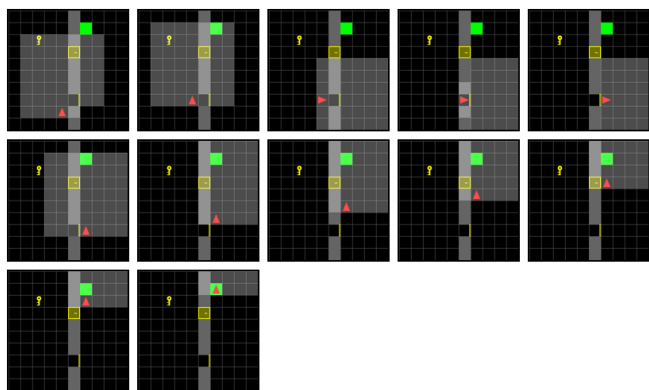
DoorKey-10x10-1



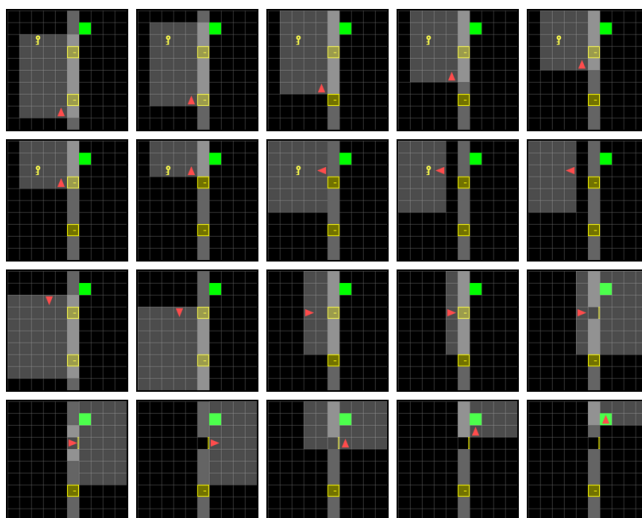
DoorKey-10x10-2



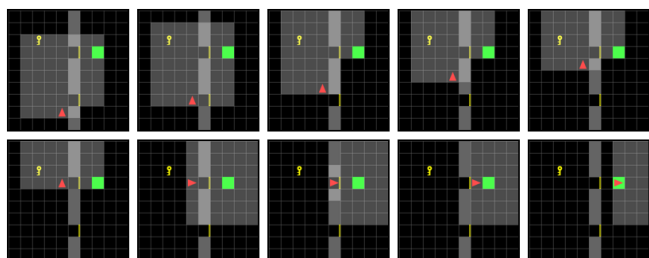
DoorKey-10x10-3



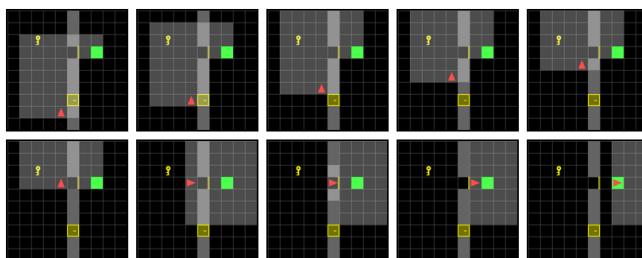
DoorKey-10x10-4



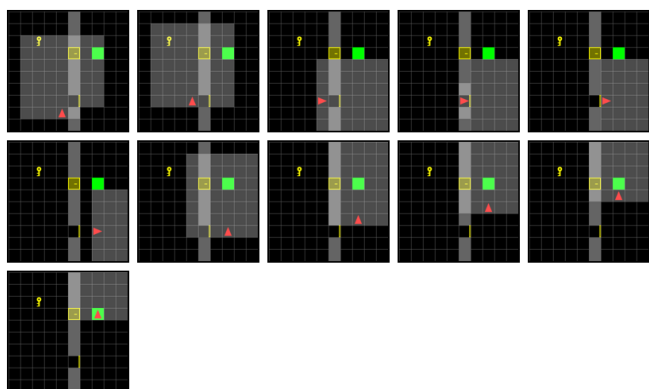
DoorKey-10x10-5



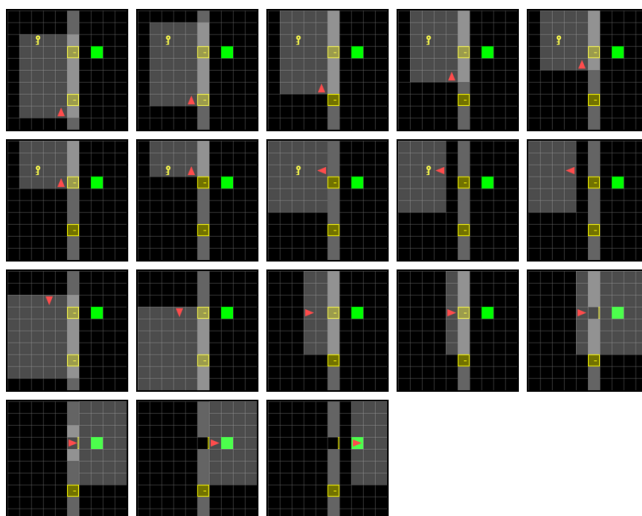
DoorKey-10x10-6



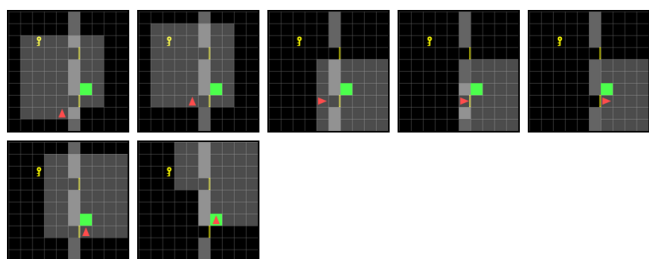
DoorKey-10x10-7



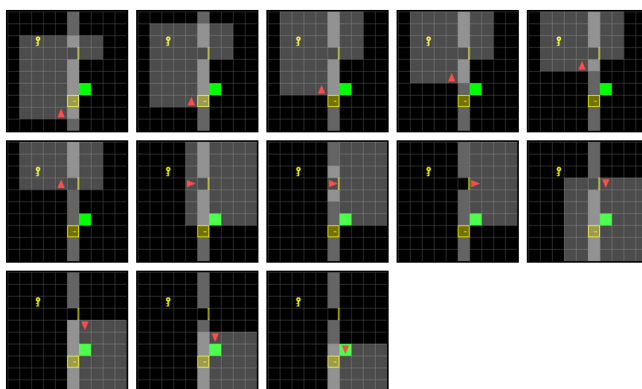
DoorKey-10x10-8



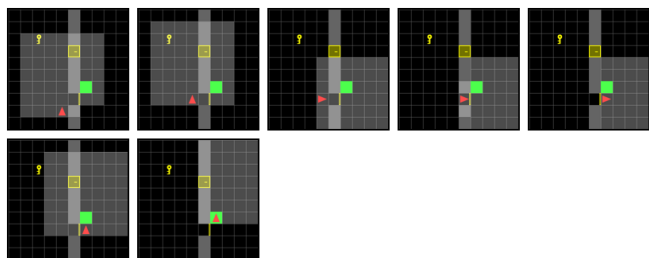
DoorKey-10x10-9



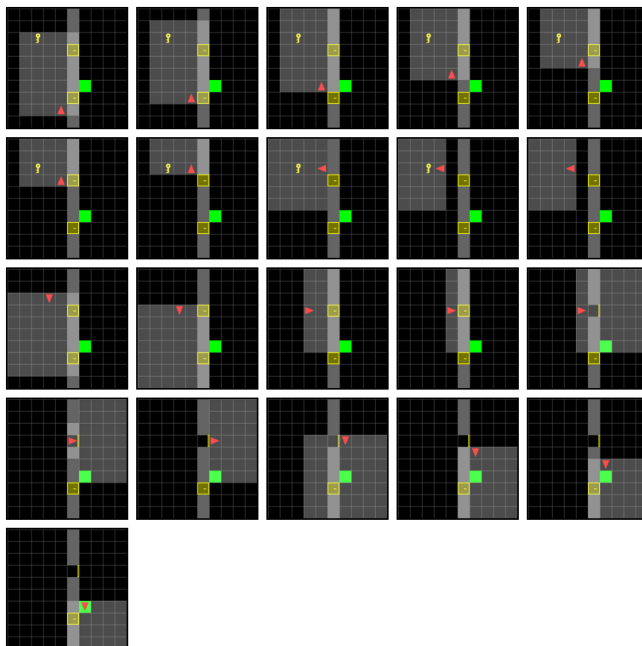
DoorKey-10x10-10



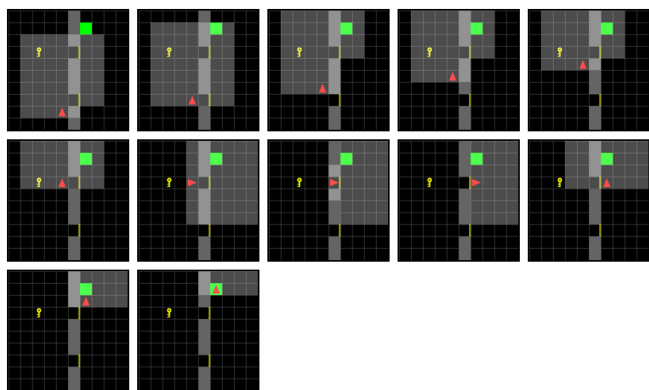
DoorKey-10x10-11



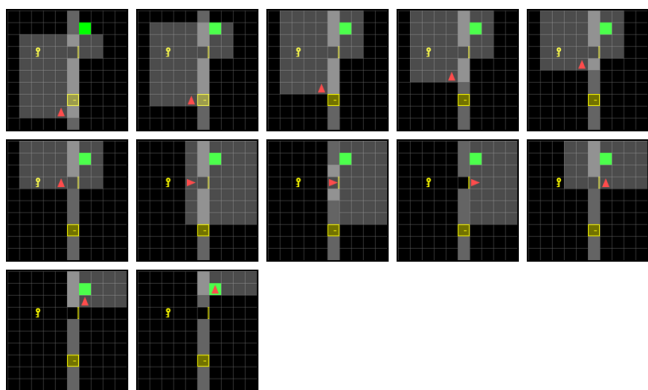
DoorKey-10x10-12



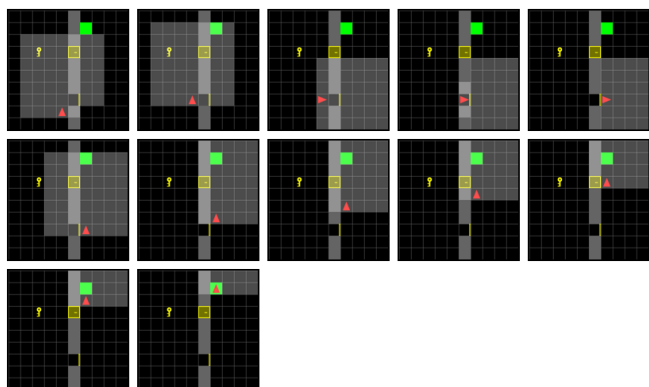
DoorKey-10x10-13



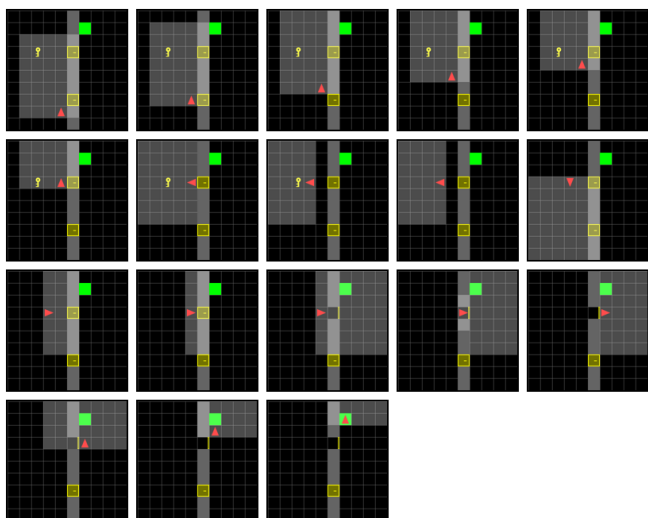
DoorKey-10x10-14



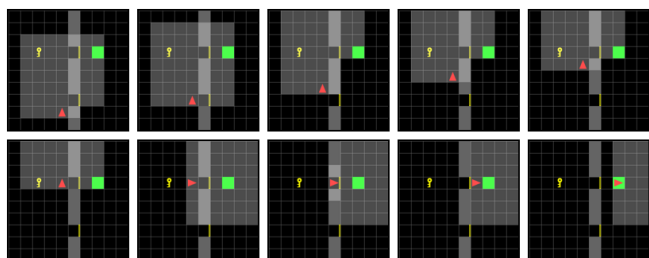
DoorKey-10x10-15



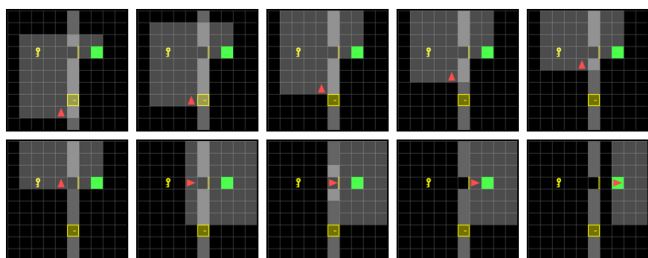
DoorKey-10x10-16



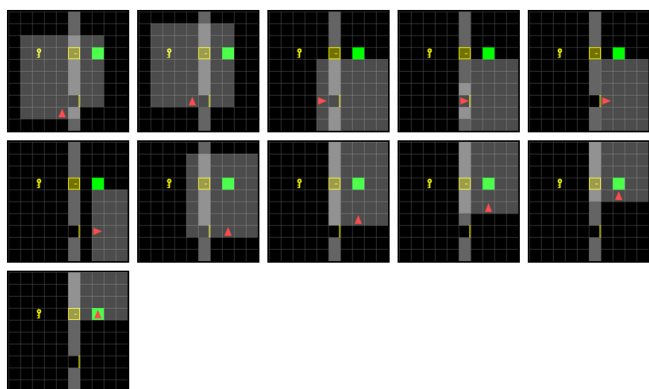
DoorKey-10x10-17



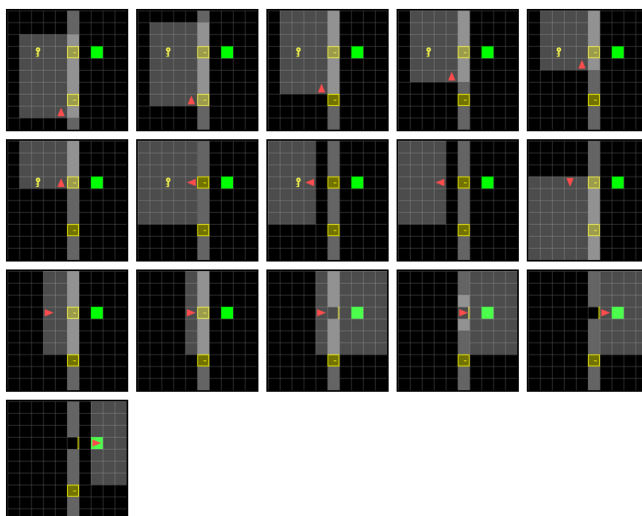
DoorKey-10x10-18



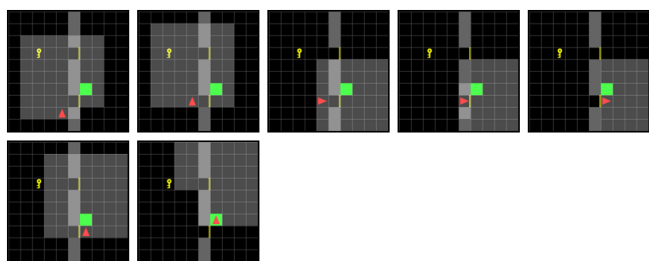
DoorKey-10x10-19



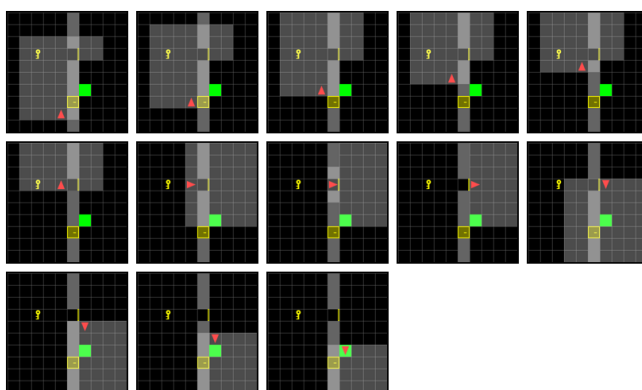
DoorKey-10x10-20



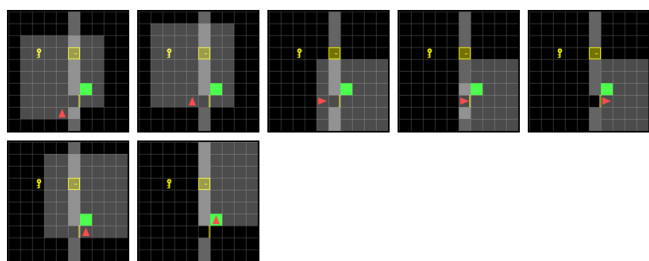
DoorKey-10x10-21



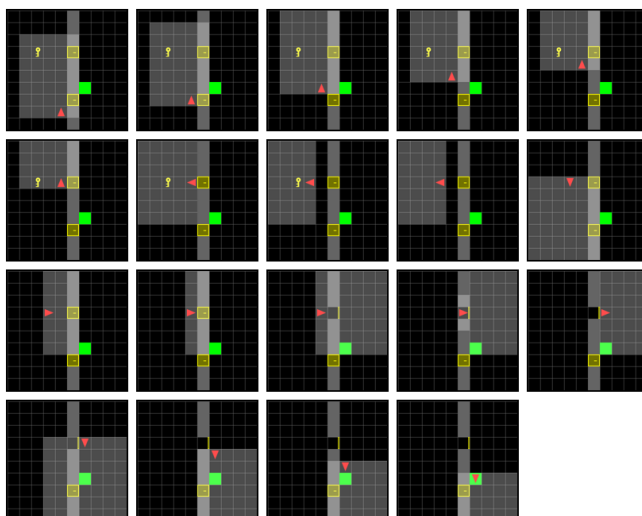
DoorKey-10x10-22



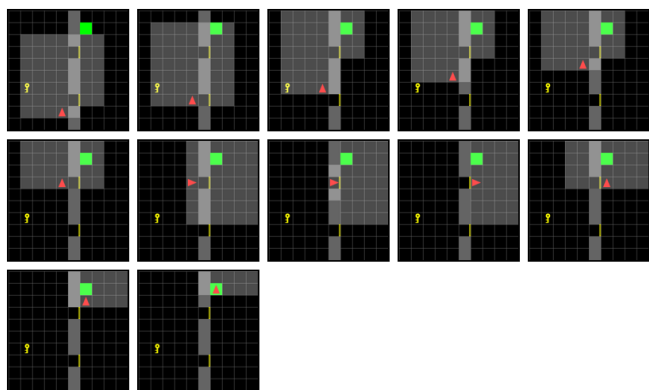
DoorKey-10x10-23



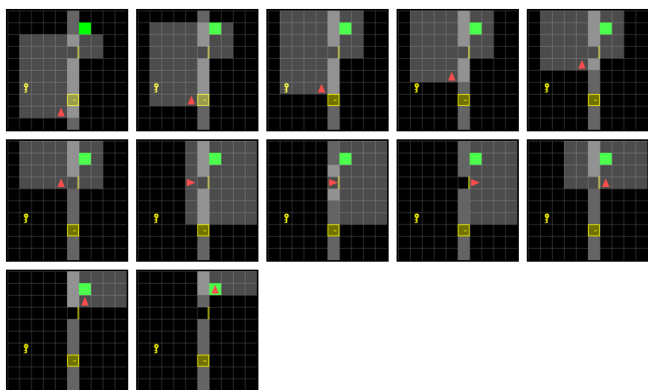
DoorKey-10x10-24



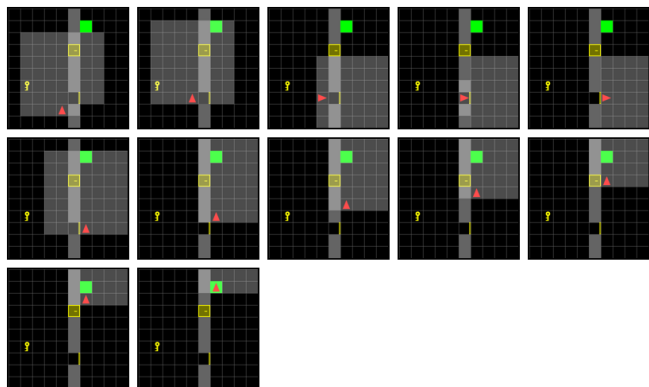
DoorKey-10x10-25



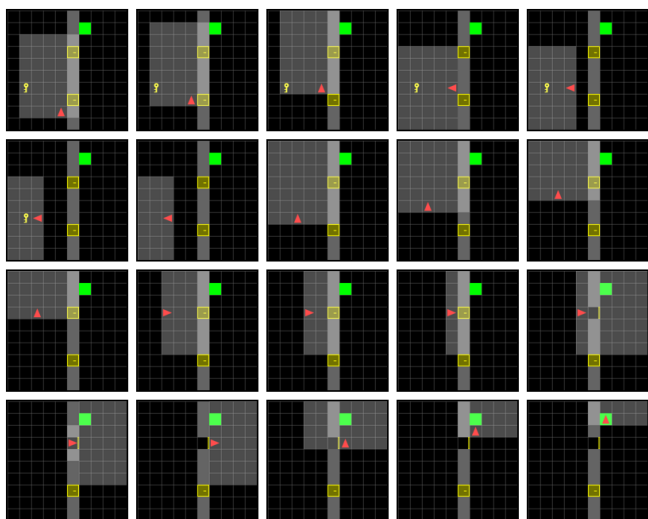
DoorKey-10x10-26



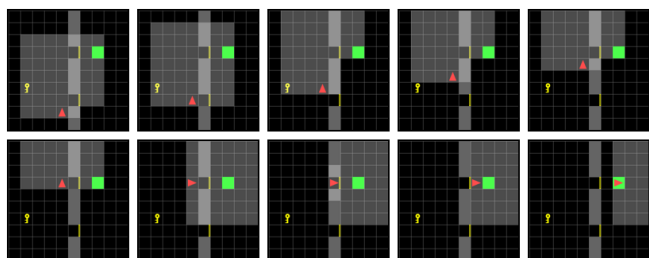
DoorKey-10x10-27



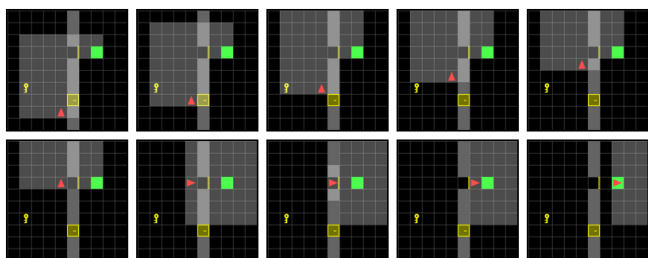
DoorKey-10x10-28



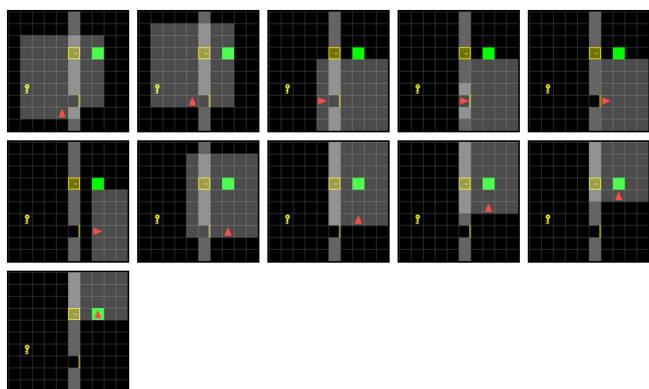
DoorKey-10x10-29



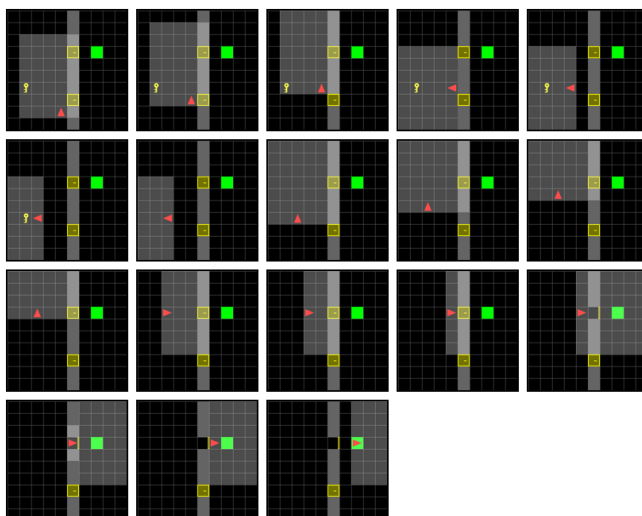
DoorKey-10x10-30



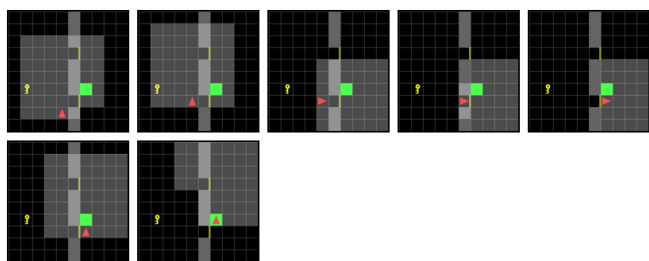
DoorKey-10x10-31



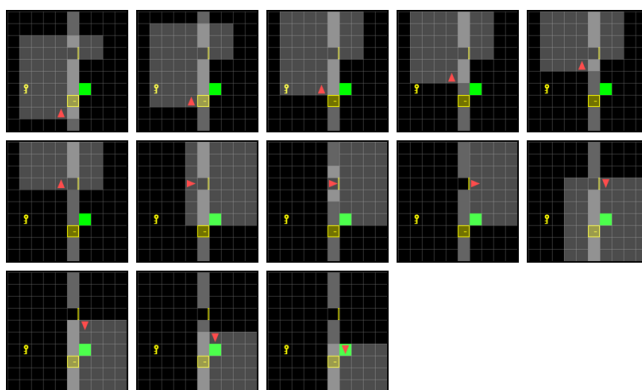
DoorKey-10x10-32



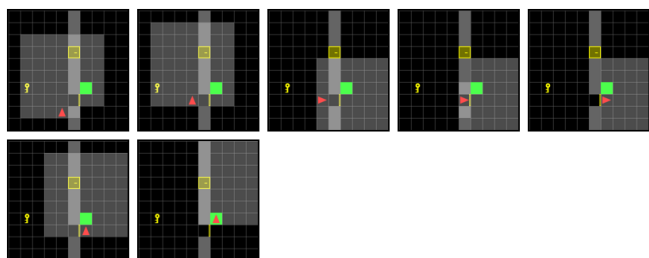
DoorKey-10x10-33



DoorKey-10x10-34



DoorKey-10x10-35



DoorKey-10x10-36

